Algoritmo e Laboratório de Programação II

Aula 9 – Ponteiros Avançados (Nível Avançado)

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

Os operadores bit-a-bit

- Vários programas precisam manipular bits individuais dentro de um byte.
- Quando mostramos os tipos de dados básicos de C, não fizemos nenhuma observação sobre como olhar dentro desses dados para saber como eles são construídos em bits individuais e como podem ser manipulados.

Os operadores bit-a-bit

- Ser capaz de operar no nível dos bits é especialmente importante quando o programa deve atuar diretamente no hardware da máquina.
- A linguagem C oferece seis operadores bit-a-bit, que permitem operar em bits individuais de variáveis do tipo char ou int.
- São eles:

Os operadores bit-a-bit

- São eles:

Símbolo	Operação
& &=	E (AND)
| |=	OU (OR)
^ ^=	OU exclusivo (XOR)
>> >>=	Deslocamento à direita (Right shift)
<< <<=	Deslocamento à esquerda (Left shift)
~ ~=	Complemento (Unário)

- Os operadores bit-a-bit operam somente com valores inteiros.

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

O operador and: & e &=

- É um operador binário que executa um AND com cada par de bits dos operandos.
- Cada bit do resultado é 1 somente quando os dois operandos são 1; em qualquer outra situação, o resultado é 0;

```
unsigned short x=5;   /* 00000000 00000101 */
unsigned short y=9;   /* 00000000 00001001 */
                      /*------------------- */
unsigned short z=x&y; /* 00000000 00000001 */
```

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

O operador or: | e |=

- Trata-se de um operador binário que executa um OR com cada par de bits dos operandos.
- Cada bit do resultado é 0 somente quando os dois bits operandos são 0; em qualquer outra situação, o resultado é 1.

```
unsigned short x=5;          /* 00000000 00000101 */
unsigned short y=9;          /* 00000000 00001001 */
                             /*------------------ */
unsigned short z=x | y;      /* 00000000 00001101 */
```

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

O operador xor: ^ e ^=

- É um operador binário que executa um XOR com cada par de bits dos operandos.
- Cada bit do resultado é 1 somente quando os dois bits operandos são diferentes; quando são iguais, o resultado é 0.

```
unsigned short x=5;    /* 00000000 00000101 */
unsigned short y=9;    /* 00000000 00001001 */
                       /*------------------ */
unsigned short z=x^y;  /* 00000000 00001100 */
```

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

O operador unário de complemento: ~ e ~=

- O operador de complemento opera um único operando; resulta um valor com todos os bits invertidos (todo bit 0 passa para 1 e todo bit 1 passa para 0).
- A aplicação do operador ~ duas vezes no mesmo operando resulta sempre o número original.
- Para mostrar o uso do operador ~ vamos escrever um programa que cria um teclado de caracteres gráficos.

O operador unário de complemento: ~ e ~=

Prog1.c

```c
/* Prog1.c */
/* Mostra o uso do operador de complemento ~ */
/* codifica os bytes digitados */
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>

int main()
{
    unsigned char ch;
    while((ch=getch())!= 'X')
        printf("%c", ~ch);
    system("pause");
    return 0;
}
```

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

O operador shift esquerdo: << e <<=

- Trata-se de um operador binário, mas que age somente no operando da esquerda.
- Executa um deslocamento para a esquerda, conforme o número de posições especificado no segundo operando.

```
unsigned short x=5;          /* 00000000 00000101 */
                             /*------------------ */
unsigned short z=x<<1;       /* 00000000 00001010 */
```

- Toda vez que empurramos um bit à esquerda multiplicamos o número por 2.

```
x <<= 1;  /* É a mesma coisa que x *= 2; */
x <<= 2;  /* É a mesma coisa que x *= 4; */
```

O operador shift esquerdo: << e <<=

- Para exemplificar o uso desse operador, vamos escrever uma função que retorna o tamanho da palavra da máquina em que o programa estiver sendo executado.

O operador shift esquerdo: << e <<=

Prog2.c

```c
/* Prog2.c */
#include <stdio.h>
#include <stdlib.h>

int palavra()
{
    unsigned int n = ~0; /* todos os bits ligados */
    int i;
    for(i = 0; n ; i++) /* enquanto n não for zero */
        n <<= 1;
    return i;
}

int main()
{
    int t = palavra();
    printf("O tamanho da palavra dessa máquina é %d bits.\n", t);
    system("pause");
    return 0;
}
```

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

O operador shift direito: >> e >>=

- É um operador binário, mas que age somente no operando da esquerda.
- Executa um deslocamento para a direita, conforme o número de posições especificado no segundo operando.

```
unsigned short x=5;       /* 00000000 00000101 */
                          /*------------------ */
unsigned short z= x>>1;   /* 00000000 00000010 */
```

- Toda vez que empurramos um bit à direita estaremos dividindo o número por 2.

```
x >>= 1; /* É a mesma coisa que x /= 2; */

x >>= 2; /* É a mesma coisa que x /= 4; */
```

O operador shift direito: >> e >>=

- Para exemplificar o uso desse operador, vamos escrever um programa que pede a entrada de um número em hexadecimal e o converte para binário.

O operador shift direito: >> e >>=

Prog3.c

```c
/* Prog3.c */
/* Converte hexa para binario */
#include <stdio.h>
#include <stdlib.h>
#include <math.h> /* para pow() */

int palavra();

int main()
{
    int t, j, num, bit;
    unsigned int mask;
    while(1)
    {
        t = palavra();

        /* um no bit mais significativo e zero nos outros */
        mask = pow(2, t-1);

        printf("\nDigite um número em hexadecimal: ");
        scanf("%x",&num) ;
        printf("Binário de %04x é: ", num);
```

O operador shift direito: >> e >>=

```
24          for(j=0; j<t ; j++)
25          {
26              bit = (mask & num) ? 1 : 0;
27              printf("%d",bit);
28              if(((j + 1) % 8) == 0) /* imprime traco entre bytes */
29                  printf("--");
30              mask >>= 1;
31          }
32      }
33      system("pause");
34      return 0;
35  }
36
37  int palavra()
38  {
39      unsigned int n = ~0; /* todos os bits ligados */
40      int i;
41      for(i = 0; n ; i++) /* enquanto n não for zero */
42          n <<= 1;
43      return i;
44  }
```

A calculadora bit-a-bit

- A variável mask tem o tamanho da palavra da maquina.
- Usamos a função de biblioteca pow() para criar uma máscara de um único bit ligado, aquele mais significativo.
- O número será impresso a partir do bit mais significativo até o bit zero.
- A cada iteração, deslocamos o bit ligado de mask uma posição à direita, e o processo de teste e impressão é repetido para o próximo bit.

Roteiro

- Os operadores bit-a-bit
- O operador and: & e &=
- O operador or: | e |=
- O operador xor: ^ e ^=
- O operador unário de complemento: ~ e ~=
- O operador shift esquerdo: << e <<=
- O operador shift direito: >> e >>=
- A calculadora bit-a-bit

A calculadora bit-a-bit

- O programa seguinte funciona como uma calculadora bit-a-bit, na qual você digita qualquer expressão simples envolvendo um operador bit-a-bit e ele imprime o resultado.

A calculadora bit-a-bit

Prog4.c

```
1  /* Prog4.c */
2  /* Calculadora bit-a-bit */
3  #include <stdio.h>
4  #include <stdlib.h>
5  void pbin(unsigned short);
6  void pline(void);
7  int main()
8  {
9      char op[3]; /* aceita dois char para deslocamento */
10     unsigned short x1, x2;
11
12     while(1)
13     {
14         printf("\n\nDigite uma expressão (ex. 'FF00 & A23C'):");
15         scanf("%x %s %x", &x1, op, &x2);
16         switch( op[0] )
17         {
18         case '&' :
19             pbin(x1);
20             printf(" & (and)\n");
21             pbin(x2);
22             pline();
23             pbin(x1 & x2);
24             break;
```

A calculadora bit-a-bit

```
25          case '|' :
26              pbin(x1);
27              printf(" | (or)\n");
28              pbin(x2);
29              pline();
30              pbin(x1 | x2);
31              break;
32          case '^':
33              pbin(x1);
34              printf(" ^ (xor)\n");
35              pbin(x2);
36              pline();
37              pbin(x1 ^ x2);
38              break;
39          case '>':
40              pbin(x1);
41              printf(" >> %d\n", x2);
42              pline();
43              pbin(x1 >> x2);
44              break;
45          case '<':
46              pbin(x1);
47              printf(" << %d\n", x2);
48              pline();
49              pbin(x1 << x2);
50              break;
```

A calculadora bit-a-bit

```
51              case '~':
52                  pbin(x1);
53                  printf(" ~ (complemento)\n");
54                  pline();
55                  pbin(~x1);
56                  break;
57              default:
58                  printf("Operador invalido.\n");
59              }
60      }
61      system("pause");
62      return 0;
63  }
```

A calculadora bit-a-bit

```
66  /* pbin() */
67  /* imprime número em binario */
68  void pbin(unsigned short num)
69  {
70      unsigned short mask;
71      short j, bit;
72      mask = 0x8000;
73      printf("%04X\t\t",num);
74      for(j=0; j<16; j++)
75      {
76          bit = (mask & num) ? 1:0;
77          printf("%d",bit);
78          if(j == 7) printf("--");
79          mask >>= 1;
80      }
81      printf("\n");
82  }
83
84  /* pline() */
85  void pline()
86  {
87      printf("--------------------");
88      printf("--------------------\n");
89  }
```

A calculadora bit-a-bit

- O programa consiste , principalmente em um comando switch que contém todos os operadores bit-a-bit.
- O usuário fornece um operando em hexadecimal, um operador bit-a-bit e um outro operando, e o programa imprime o resultado.

A calculadora bit-a-bit

- A única irregularidade é com o operador de complemento, que é unário, e o programa solicita um segundo operando para satisfazer a função scanf() (qualquer número pode ser digitado).

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;